

MATH 448 - Term Project Report

Classification of Edible vs. Poisonous Mushrooms

Elizabeth Guzman Ramirez

December 7, 2025

MATH 448 - Term Project Report.....	1
Executive Summary.....	3
1. Introduction.....	4
2. Data Description, Cleaning, and Preparation.....	4
2.1 Dataset Overview.....	4
2.2 Missing Values.....	5
2.3 Dummy Variable Encoding.....	5
2.4 Train–Test Split.....	5
3. Data Visualization.....	6
3.1 Class Balance.....	6
3.2 Odor vs. Class.....	7
3.3 Bruises vs. Class.....	8
3.4 Gill-Size vs. Class.....	8
4. Methodology and Results.....	9
4.1 Logistic Regression.....	9
4.2 K-Nearest Neighbors (KNN).....	10
4.3 Decision Tree.....	10
4.4 Naive Bayes.....	11
4.5 Quadratic Discriminant Analysis (QDA).....	12
4.6 Bagging (Bootstrap Aggregation).....	12
Model Comparison Summary.....	13
5. Conclusion and Future Work.....	13
References.....	14
Appendix.....	15

Executive Summary

Distinguishing mushrooms from one another can be a difficult task. Many edible and poisonous varieties look nearly identical, which makes accurate identification challenging. Relying on visual traits alone can be risky and harmful to the public, so the ability to classify mushrooms using statistical learning is valuable. By analyzing patterns across their recorded characteristics, we can build models that help distinguish safe mushrooms from harmful ones.

The UCI Mushroom Dataset gave me an opportunity to explore this problem in a structured way. The dataset contains 8,124 mushrooms, each described by 22 categorical features such as cap color, odor, gill size, and habitat, along with a label indicating whether the mushroom is edible or poisonous. Although the dataset does not include species information, it allows us to study how strongly different physical traits relate to toxicity and how various statistical learning methods perform on a classification task where patterns are present but not immediately obvious to the human eye.

The goal of this project was to build, compare, and interpret several supervised learning models to predict mushroom toxicity. After removing one feature with missing data and converting all remaining categorical predictors into dummy variables, I split the data into training and testing sets using a fixed seed for reproducibility. I applied the methods we learned in MATH 448: Logistic Regression, K-Nearest Neighbors, Decision Trees, Naive Bayes, Quadratic Discriminant Analysis, and Bagging.

The results were very strong across all models. Most methods gave almost perfect accuracy such as KNN, Decision Trees, QDA, and Bagging correctly classified every mushroom in the test set. Even the more restrictive models, such as Logistic Regression and Naive Bayes, performed above 97%. These outcomes suggest that certain mushroom characteristics, like odor and gill-related features, contain enough information to separate edible and poisonous mushrooms with extremely high accuracy.

Overall, this project highlights how effective classical statistical learning methods can be when the predictors carry strong, meaningful patterns related to the outcome. It also reinforces the importance of foundational steps such as cleaning the data, encoding categorical variables, choosing appropriate models, and evaluating them consistently. While identifying mushrooms in real life is much more complex, this dataset provides a clear example of how supervised learning can perform when the structure of the data aligns well with the classification task.

1. Introduction

The dataset used in this project comes from the UCI Machine Learning Repository, which has a collection of real-world datasets for research and teaching in statistical learning. The mushroom dataset was originally compiled from entries in The Audubon Society Field Guide to North American Mushrooms, where thousands of mushroom specimens were documented along with physical characteristics such as cap color, odor, gill structure, and habitat. The goal of the dataset is to provide a standardized reference for studying mushroom identification and toxicity.

The dataset includes 8,124 mushrooms described by 22 categorical features, along with a label indicating whether each mushroom is edible or poisonous. Although species names are not included, the dataset still works well for this project because many edible and poisonous mushrooms share similar characteristics, making classification meaningful.

In this project, I explored how well several supervised learning methods from MATH 448 can classify mushroom toxicity based on these recorded characteristics. After preprocessing the dataset, removing one feature with substantial missing values and converting all categorical variables into dummy indicators, I applied Logistic Regression, K-Nearest Neighbors, Decision Trees, Naive Bayes, Quadratic Discriminant Analysis, and Bagging. The goal of this analysis was to compare these models and understand how classical statistical learning techniques perform on a dataset where meaningful patterns are present.

2. Data Description, Cleaning, and Preparation

2.1 Dataset Overview

The UCI Mushroom Dataset has 8,124 observations, each representing a single mushroom described by 22 categorical attributes. These attributes capture physical characteristics such as cap shape, cap color, odor, gill size, gill spacing, habitat, and more. The response variable, class, indicates whether each mushroom is edible (0) or poisonous (1).

The dataset is nearly balanced, with approximately 52% edible and 48% poisonous mushrooms. Because the two classes appear in similar proportions, accuracy is a reasonable and reliable metric for evaluating model performance.

2.2 Missing Values

The dataset is almost entirely complete, with only one feature containing missing entries. The stalk-root variable had many blank values, and because this missingness did not provide useful information and the feature was unlikely to contribute meaningfully to the classification task, I removed it during data cleaning. All other features were kept for analysis.

2.3 Dummy Variable Encoding

All of the predictors in the dataset are categorical and stored as letters, so before fitting any models, I converted them into dummy variables. This process creates a separate 0/1 column for each category, allowing the models to use the information correctly without treating the categories as numerical values. I used `drop_first=True` to avoid creating redundant columns. After this step, the full feature matrix contained 91 dummy-coded predictors.

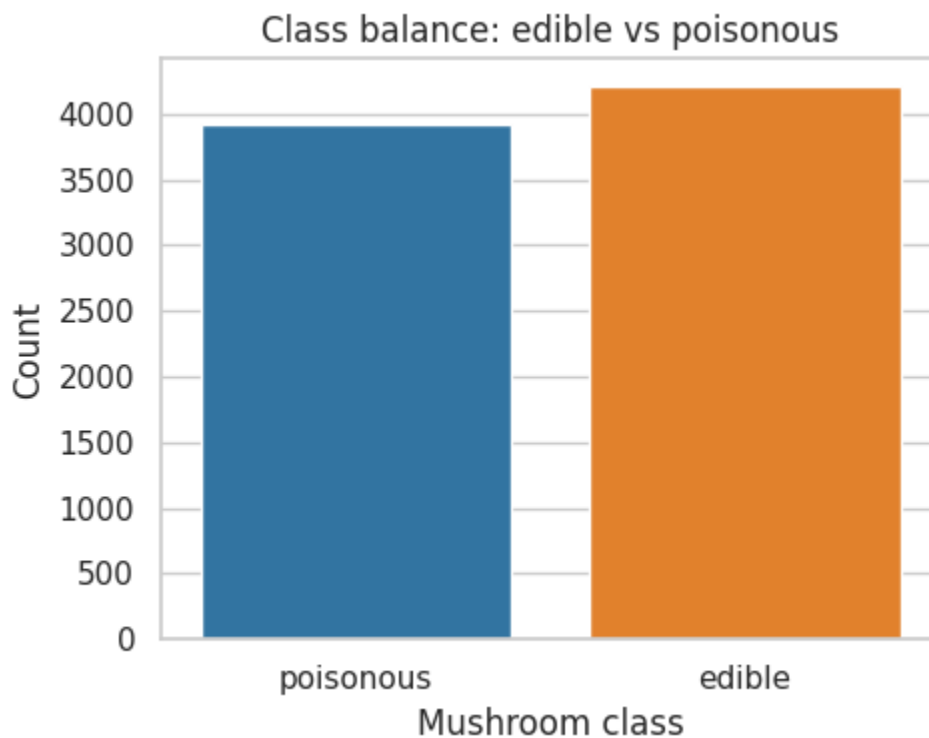
2.4 Train–Test Split

I divided the dataset into a training set (70%) and a testing set (30%) using a fixed random seed (448) to make the split reproducible. I also used the stratify option so that the edible and poisonous classes remained in the same proportions in both sets. This helps maintain consistency in how the models learn from and are evaluated on the data.

3. Data Visualization

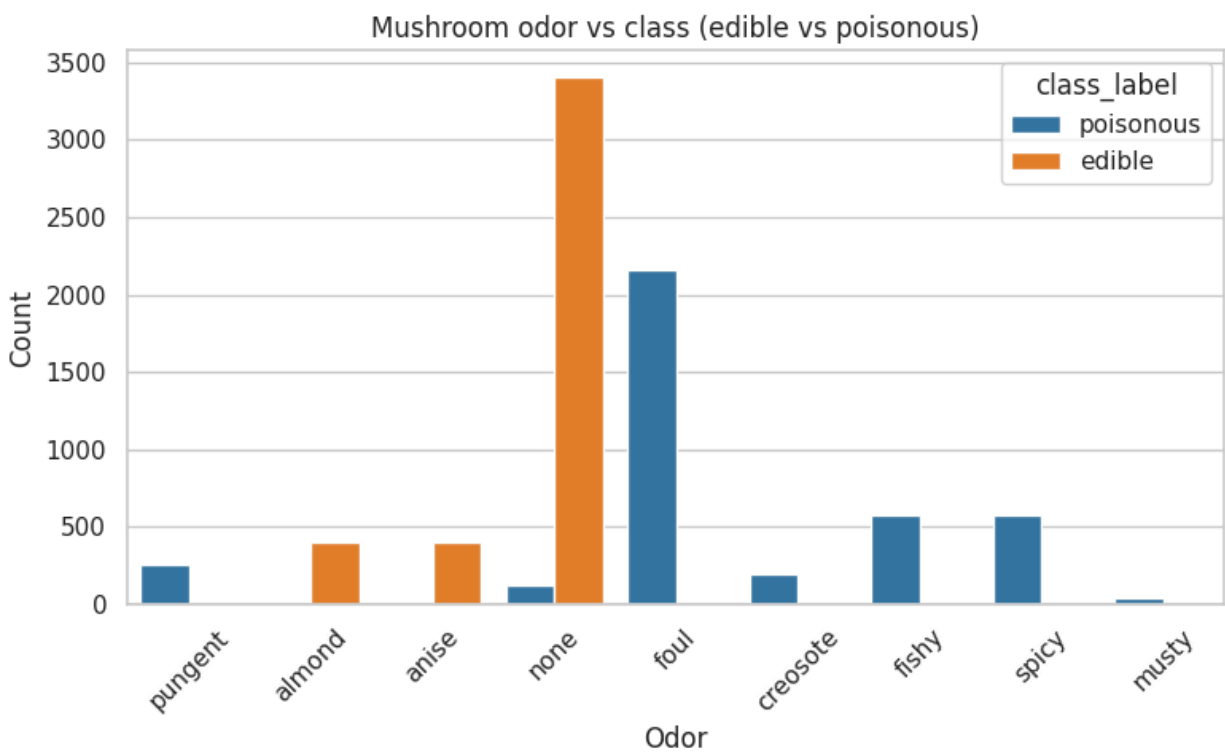
3.1 Class Balance

Before fitting any models, I checked the overall class balance in the dataset. The bar plot of edible vs. poisonous mushrooms shows that the two classes are almost equal in size. The dataset contains 4,208 edible mushrooms and 3,916 poisonous mushrooms, which is very close to a 50/50 split. Because neither class dominates the other, the models are not expected to be biased toward predicting one outcome more often. This balanced structure also makes accuracy an appropriate and meaningful evaluation metric for this classification problem.



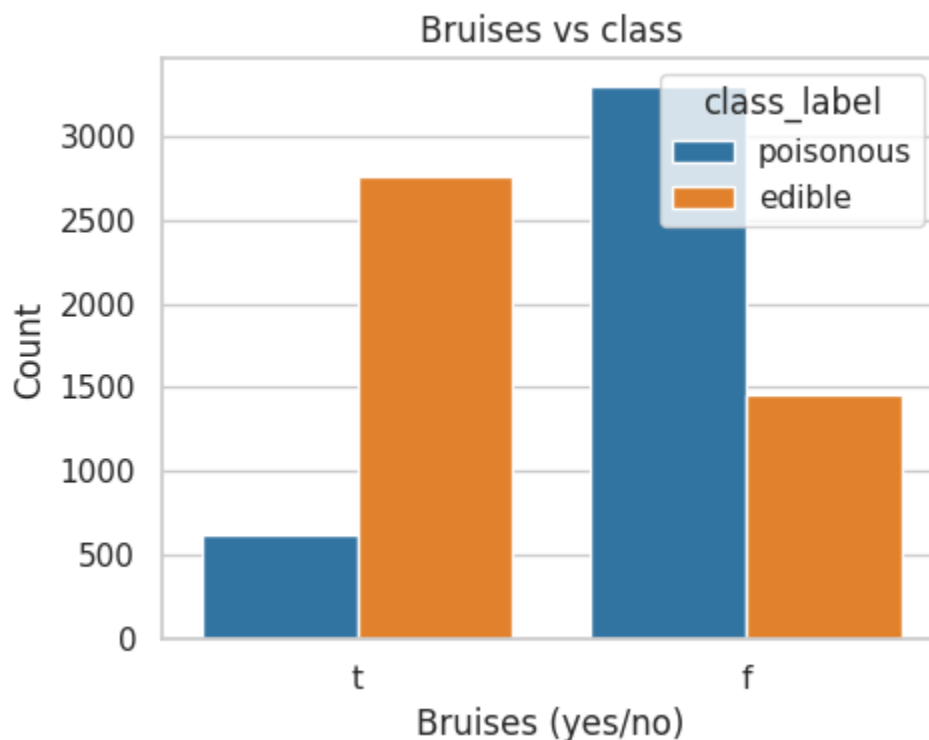
3.2 Odor vs. Class

Odor turned out to be one of the strongest visual indicators of mushroom toxicity. The plot shows a clear separation between edible and poisonous mushrooms based on their smell. Most poisonous mushrooms have distinct odors such as foul, creosote, fishy, spicy, or pungent. By contrast, almost all edible mushrooms have no odor at all, with smaller groups having mild smells like almond or anise. This dramatic difference suggests that odor is a highly informative predictor for classification and it helps explain why many of the models later achieved near-perfect accuracy.



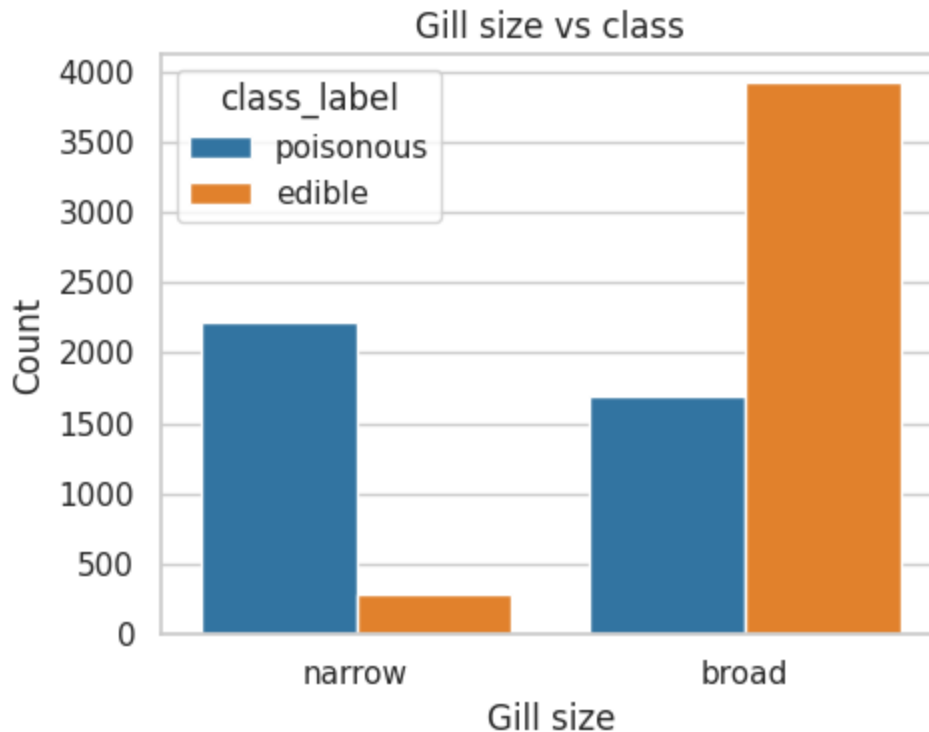
3.3 Bruises vs. Class

The bruises feature shows a weaker relationship with toxicity compared to odor. In the plot, most poisonous mushrooms fall into the f (no bruises) category, while edible mushrooms are more evenly split but still lean toward t (bruises). This pattern indicates that the bruises variable does carry useful information for classification, but its predictive strength is limited compared to more decisive features like odor.



3.4 Gill-Size vs. Class

Gill size shows a very strong relationship with mushroom toxicity. The plot reveals that most poisonous mushrooms have narrow gills, while nearly all edible mushrooms have broad gills. This separation is much more distinct than what we saw with bruises. Because the two classes separate so clearly by gill size, this feature is likely to be one of the most influential predictors in the entire dataset.



4. Methodology and Results

4.1 Logistic Regression

Logistic Regression was the first model I fit for this project. Since the outcome is binary (edible = 0, poisonous = 1), this method works by modeling the probability that a mushroom is poisonous based on all the dummy-coded predictors. After cleaning the data and creating the training/testing split, I fit the model using the training set and then checked how well it predicted both the training and test mushrooms. Because the classes are already balanced, accuracy is a reasonable way to measure how well the model performs.

Logistic Regression performed much better than I expected. The model got 99.96% of the training mushrooms correct and 99.92% of the test mushrooms correct. In the test set, only two mushrooms were misclassified out of 2,438. The confusion matrix clearly shows how well the model separates the edible mushrooms from the poisonous ones. Even though logistic regression is one of the more restrictive models I used, because it can only capture linear relationships, it

still did extremely well on this dataset. This suggests that the mushroom features are already very informative, and even a simple model can separate the two classes with almost perfect accuracy.

4.2 K-Nearest Neighbors (KNN)

K-Nearest Neighbors looks at the k closest observations in the training data and predicts the majority class among them. Because KNN is entirely distance-based, all the predictors have to be numeric or encoded, which is why the dummy-variable step was important earlier. KNN can capture more flexible, nonlinear relationships than logistic regression, but its performance depends on the choice of k . A very small k can overfit, while a very large k can oversmooth the patterns. For consistency, I followed the Chapter 4 lab code and used $k = 5$, which provides a good balance between flexibility and stability.

KNN with $k = 5$ performed extremely well on the mushroom dataset. Both the training and test accuracy were 100%, and the model correctly classified every single mushroom in the test set. Because KNN classifies each mushroom by looking at its closest neighbors, the perfect accuracy suggests that edible and poisonous mushrooms end up in very separate regions of the predictor space after encoding. In other words, once the features are turned into dummy variables, the two classes are easy for KNN to distinguish.

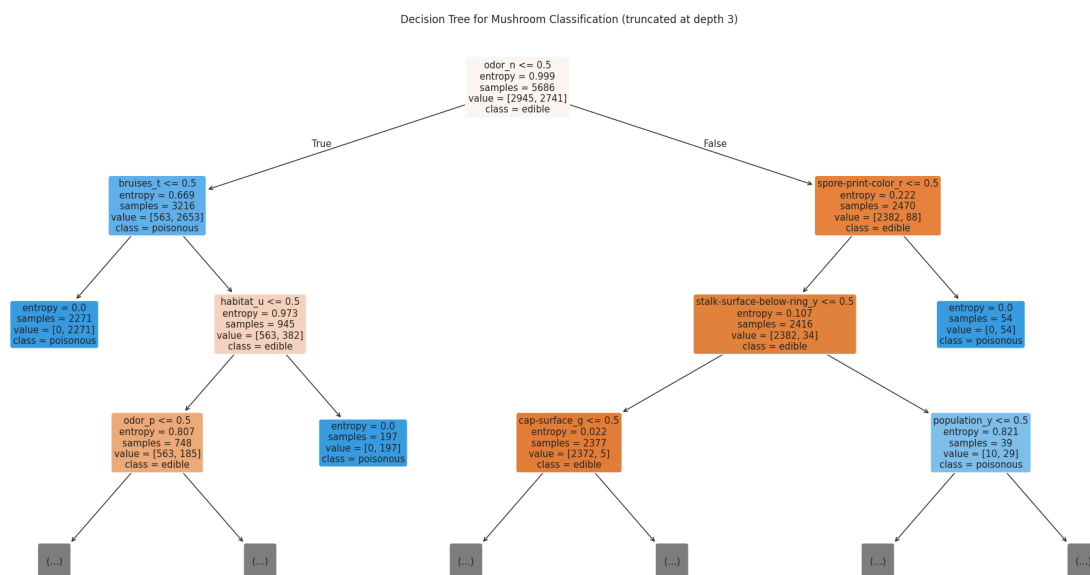
4.3 Decision Tree

I fit a Decision Tree classifier using the same X and y training sets as before, following the structure of the earlier models. Decision Trees work by splitting the data into smaller and smaller groups based on rules that try to separate the classes as clearly as possible. In my model, each split chooses the feature that best divides edible and poisonous mushrooms. I followed the approach from the Chapter 4 lab code and used the default settings, keeping the same seed for reproducibility.

To better understand how the model makes decisions, I also plotted the tree and truncated it at depth 3 (shown in Figure 5). In the figure, we can see that the tree immediately splits on odor, which lines up with the strong patterns seen in the odor vs. class plot. Mushrooms with odors

like foul, pungent, or creosote almost always fall into the poisonous class, while odors such as none, almond, and anise are almost entirely edible. After that, the tree uses features like bruises, habitat, stalk-surface, and gill characteristics to refine the groups even further.

When I evaluated the Decision Tree on the test data, the model correctly classified every single mushroom. The confusion matrix had all observations along the diagonal and zero misclassifications, meaning the tree made no errors on either class. Even though decision trees often risk overfitting, in this case the structure of the mushroom dataset is strong enough that a single tree can perfectly separate the two classes.



4.4 Naive Bayes

I fit a Naive Bayes classifier using the same training data as before. Naive Bayes assumes that all mushroom features are independent once we know whether the mushroom is edible or poisonous. This isn't actually true for this dataset, since many traits are related, but the method still works surprisingly well when the patterns in the data are strong.

After fitting the model on the dummy encoded predictors and evaluating it on the test set, I found that Naive Bayes performed very well. The test accuracy was about 97.99%, meaning it correctly classified the vast majority of mushrooms.

The confusion matrix showed that the model made only a few mistakes: it misclassified 48 edible mushrooms as poisonous, and only one poisonous mushroom as edible. This last mistake is the most concerning type of error, but it only happened once out of more than 1,100 poisonous samples.

Overall, Naive Bayes handled this dataset really well. Even though it didn't reach the perfect accuracy of some of the other models, it still captured the major patterns, especially the strong signals from odor and gill-related features. It's a great example of how a simple model can still perform at a high level, even when its assumptions don't fully match how the data actually behaves.

4.5 Quadratic Discriminant Analysis (QDA)

For this model, I fit a Quadratic Discriminant Analysis (QDA) classifier using the same training data as before. QDA is similar to Linear Discriminant Analysis, but it's more flexible because it lets each class (edible vs. poisonous) have its own covariance structure. Basically, this means QDA can draw curved or nonlinear boundaries between the classes, which can help when the split between groups isn't perfectly straight.

After fitting the model and testing it, QDA ended up with perfect accuracy. It correctly classified every mushroom in the test set, with the confusion matrix showing zero mistakes, meaning every edible mushroom was labeled edible, and every poisonous mushroom was labeled poisonous.

The model did give a small numerical warning about the covariance matrix not being full rank. That usually happens when some predictors are highly correlated, especially with lots of dummy variables. But in this case, the warning didn't impact the results at all. The dataset is so cleanly separated that QDA was still able to find a perfect decision boundary.

Overall, QDA performed just as well as KNN, Decision Trees, and Bagging. It's another example of how a more flexible method can take advantage of the strong patterns in this dataset and separate the two classes perfectly.

4.6 Bagging (Bootstrap Aggregation)

For this model, I used Bagging (Bootstrap Aggregation) by training a `BaggingClassifier` with 200 bootstrap samples (trees). Bagging works by building a bunch of decision trees on different random chunks of the data and then averaging their predictions. A single tree can be a little unstable on its own, but combining many of them usually smooths things out and gives better accuracy.

After training the model on the same X and y data, I tested it and found that Bagging correctly classified every single mushroom. Just like KNN, QDA, and the decision tree, it didn't make any mistakes. The confusion matrix had all observations on the diagonal, meaning there were zero misclassifications in either class.

This result makes sense for this dataset. The patterns separating edible and poisonous mushrooms are so strong that even the individual trees in the ensemble already perform extremely well. Once their predictions are averaged together, the model becomes even more stable while still capturing the same clear structure.

Overall, Bagging performed just as well as the other top models. It shows how combining many trees can create a highly accurate and reliable classifier, especially when the features in the dataset already separate the classes so clearly.

Model Comparison Summary

Logistic Regression and Naive Bayes were the only methods that made any errors, and even those were small, both stayed above 97% accuracy. The more flexible models, like KNN, Decision Trees, QDA, and Bagging, reached perfect accuracy and didn't misclassify a single mushroom in the test set. This really highlights how strong and consistent the patterns are in the mushroom data. Features like odor, gill size, and bruising give such clear signs that almost any reasonable model can pick up the difference between edible and poisonous mushrooms. In the end, no single model stands out as "the best" because they all performed at a very high level, they just arrived at the same conclusion in slightly different ways.

5. Conclusion and Future Work

This project explored a variety of supervised learning methods to classify mushrooms as edible or poisonous using the UCI Mushroom Dataset. After cleaning and encoding the data, I applied several models such as Logistic Regression, KNN, Decision Trees, Naive Bayes, QDA, and Bagging. Many of these models achieved near-perfect or perfect accuracy. Seeing these results made it clear that the mushroom features in this dataset line up almost perfectly with the class labels, which is why even the simpler methods performed extremely well.

Even though this dataset is great for practicing model building, it's important to remember that it was designed specifically for teaching and student learning. The features are clean, categorical, and intentionally well-separated so beginners can focus on understanding model behavior without dealing with messy or ambiguous data. Now that I've worked through this dataset and seen how different methods behave when the patterns are strong, the next step would be to try a more complex and realistic dataset. A real mushroom dataset might include measurements (such as cap diameter or stem height), geographic or environmental information, multiple samples per species, growth stages, or even laboratory test results. These kinds of features add variability and noise, making the classification problem much closer to real-world mushroom identification. It would also be interesting to explore more advanced models like Random Forests or Boosting to see whether they offer any improvements, even though the simpler methods already performed exceptionally well here.

Overall, this project helped me understand these learning techniques more deeply, and now I'm excited to apply them to datasets that are more complex and less "classroom-friendly."

References

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2023). *An introduction to statistical learning: With applications in Python* (2nd ed.). Springer. <https://www.statlearning.com/>

Kuhn, M., & Johnson, K. (2020). *ISLP Python Labs: Chapter 4 — Classification lab* [Jupyter notebook]. GitHub.

https://github.com/intro-stat-learning/ISLP_labs/blob/stable/Ch04-classification-lab.ipynb

Kuhn, M., & Johnson, K. (2020). *ISLP Python Labs: Chapter 8 — Bagging and boosting lab* [Jupyter notebook]. GitHub.

https://github.com/intro-stat-learning/ISLP_labs/blob/stable/Ch08-baggboost-lab.ipynb

Dua, D., & Graff, C. (2019). *Mushroom* [Dataset]. UCI Machine Learning Repository.

<https://archive.ics.uci.edu/dataset/73/mushroom>

Appendix

```
#Elizabeth Guzman Ramirez
#Professor Tao He
#MATH 448 - Tue/Thur

#I am importing packages

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.tree import DecisionTreeClassifier, plot_tree

#Seed for consistent randomness
seed = 448

sns.set(style="whitegrid")

#UCI Mushroom data set : Loading
mushroom = fetch_ucirepo(id=73)

#Note to self, "mushroom" (right above) is an object that holds
#input variables and the response.

# Predictor (X) columns (gills, color, smell, etc)
#Also just made a copy of the link so I can manipulate the data set
X = mushroom.data.features.copy()

# Response (y) edible or poisonous
#remember that ".copy" makes a copy so I can manipulate the data set
```

```

y = mushroom.data.targets.copy()

#Let's look at the data.
#Remember that ".shape" give us the dimension of the data Frame
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

print("\nFirst 5 rows of X (no row numbers):")
print(X.head().to_string(index=False))

#FYI, "poisonous" will print out on top b/c its the column title.
#It's not saying everything is poisonous.
#This is cheking to see how many mushrooms are edible vs. poisonous.
print("\nResponse value counts:")
print(y.value_counts())

#Now that we know the shape and size of the data set, let's check it for any
#missing values
print("\nNumber of missing values in each feature:")
print(X.isna().sum())

print("\nAny missing values in target?")
print(y.isna().sum())

#Double checking to see if there are 2470 missing values
X.isna().any().any()

#Doble checking to see if "stalkroot" specifically has these missing values
X[X['stalk-root'].isna()].head()

```

```

# So far, I loaded the mushroom dataset and checked its shape, the first few
# rows, the edible/poisonous counts, and any missing values
# (only stalk-root had missing data). Now I'm going to clean and encode the
# data
# so the machine learning models can use it.

# Reset X and y from the original mushroom object,
# so this block works even if we re-run it.
X = mushroom.data.features.copy()
y = mushroom.data.targets.copy()

# We're not going to need the "stalk-root" column because the whole thing is
# missing data. This command will get rid of the column.
X = X.drop(columns=["stalk-root"])

```

```

# This way I can double check that it was actually dropped
print("\nColumns after dropping 'stalk-root':")
print(X.columns)

# All predictors are categorical and stored as letters.
# To use them in a model, I need to convert each category
# into its own 0/1 dummy column.
# "get_dummies()" handles this automatically,
# and "drop_first=True" avoids extra duplicate columns.
X_dummies = pd.get_dummies(X, drop_first=True)

print("\nShape of X_dummies after encoding (0/1):", X_dummies.shape)

# y is currently a DataFrame with 1 column.
# I want a single 1D column (a Series), so I pull out that column.
y = y.iloc[:, 0]

# In the original data: 'e' = edible, 'p' = poisonous
# Here: edible = 0, poisonous = 1
y_dummies = y.map({"e": 0, "p": 1})

print("\nResponse value counts after encoding (0 = edible, 1 = poisonous):")
print(y_dummies.value_counts())

# I split the dataset into a training set (70%) and a testing set (30%).
X_train, X_test, y_train, y_test = train_test_split(
    X_dummies,
    y_dummies,
    test_size=0.30,
    random_state=seed,    # uses seed = 448 from above

    # stratify=y_dummies keeps the edible/poisonous ratio the same in both
    # the training and testing sets. This prevents an unbalanced split.
    stratify=y_dummies
)

print("\nTrain / test shapes:")
print("X_train:", X_train.shape)
print("X_test: ", X_test.shape)
print("y_train:", y_train.shape)
print("y_test: ", y_test.shape)

```

```

#Now that the data is cleaned, I want to look at the data before the model

```

```

# For plots, I'll go back to the original categorical X and y
mushroom = fetch_ucirepo(id=73)
X_viz = mushroom.data.features.copy()
y_viz = mushroom.data.targets.copy()

# Make one DataFrame that includes predictors and the original class labels
viz_df = X_viz.copy()
viz_df["class"] = y_viz["poisonous"] # column name is "poisonous" in y

# Map class codes to readable labels
class_map = {"e": "edible", "p": "poisonous"}
viz_df["class_label"] = viz_df["class"].map(class_map)

# Custom color palette: bright blue + orange
custom_colors = ["#1f77b4", "#ff7f0e"] # edible, poisonous

# 6.1 Bar plot of overall class balance (edible vs poisonous)
plt.figure(figsize=(5, 4))
sns.countplot(x="class_label", data=viz_df, palette=custom_colors)
plt.xlabel("Mushroom class")
plt.ylabel("Count")
plt.title("Class balance: edible vs poisonous")
plt.tight_layout()
plt.show()

#-----Odor vs class-----#
odor_map = {
    "a": "almond",
    "l": "anise",
    "c": "creosote",
    "f": "foul",
    "m": "musty",
    "n": "none",
    "p": "pungent",
    "s": "spicy",
    "y": "fishy"
}

viz_df["odor_full"] = viz_df["odor"].map(odor_map)

plt.figure(figsize=(8, 5))
sns.countplot(x="odor_full", hue="class_label", data=viz_df,
palette=custom_colors)
plt.xlabel("Odor")

```

```

plt.ylabel("Count")
plt.title("Mushroom odor vs class (edible vs poisonous)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

#-----Bruises vs Class-----#
plt.figure(figsize=(5, 4))
sns.countplot(x="bruises", hue="class_label", data=viz_df,
palette=custom_colors)
plt.xlabel("Bruises (yes/no)")
plt.ylabel("Count")
plt.title("Bruises vs class")
plt.tight_layout()
plt.show()

#-----Gill-size vs Class-----#
gill_map = {"b": "broad", "n": "narrow"}
viz_df["gill_size_full"] = viz_df["gill-size"].map(gill_map)

plt.figure(figsize=(5, 4))
sns.countplot(x="gill_size_full", hue="class_label", data=viz_df,
palette=custom_colors)
plt.xlabel("Gill size")
plt.ylabel("Count")
plt.title("Gill size vs class")
plt.tight_layout()
plt.show()

```

```

#----- Methodology and Results part -----#

# I am using Logistic Regression as my first classification model.
# It models the probability that a mushroom is poisonous (1) vs edible (0).

log_model = LogisticRegression(
    max_iter=1000,      # allow more iterations so it can converge
    random_state=seed   # use the same seed = 448 for reproducibility
)

# Fit the model on the training data
log_model.fit(X_train, y_train)

# Predictions on training and test sets
y_train_pred_log = log_model.predict(X_train)

```

```

y_test_pred_log = log_model.predict(X_test)

# Classification accuracy on training and test data
train_acc_log = accuracy_score(y_train, y_train_pred_log)
test_acc_log = accuracy_score(y_test, y_test_pred_log)

print("\nLogistic Regression (Model #1)")
print("Training accuracy:", train_acc_log)
print("Test accuracy    :", test_acc_log)

# Confusion matrix on the test data
print("\nConfusion matrix for Logistic Regression (test data):")
print(confusion_matrix(y_test, y_test_pred_log))

#Some notes after running (for my write-up later):
#On the training set, logistic regression correctly classified
#about 99.96% of mushrooms.
#On the test set, it correctly classified about 99.92% of mushrooms.

```

```

# Model 2 : K-Nearest Neighbors

# I'll start with k = 5
k = 5

# Set up the KNN classifier
knn = KNeighborsClassifier(n_neighbors=k)

# Fit KNN on the training data
knn.fit(X_train, y_train)

# Predictions on training and test sets
y_train_pred_knn = knn.predict(X_train)
y_test_pred_knn = knn.predict(X_test)

# Training and test accuracy
train_acc_knn = accuracy_score(y_train, y_train_pred_knn)
test_acc_knn = accuracy_score(y_test, y_test_pred_knn)

print("K-Nearest Neighbors (Model #2)")
print(f"k = {k}")

```

```

print("Training accuracy:", train_acc_knn)
print("Test accuracy      :", test_acc_knn)

# Confusion matrix on test data
cm_knn = confusion_matrix(y_test, y_test_pred_knn)
print("\nConfusion matrix for KNN (test data):")
print(cm_knn)

```

```

# ----- Decision Tree (Model #3) -----#

# I am using a classification tree to predict poisonous (1) vs edible (0).
# Using 'entropy' to match the lecture code.

tree_model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=seed # same seed = 448 for reproducibility
)

# Fit the tree on the training data
tree_model.fit(X_train, y_train)

# Predictions on training and test sets
y_train_pred_tree = tree_model.predict(X_train)
y_test_pred_tree  = tree_model.predict(X_test)

# Classification accuracy on training and test data
train_acc_tree = accuracy_score(y_train, y_train_pred_tree)
test_acc_tree  = accuracy_score(y_test, y_test_pred_tree)

print("\nDecision Tree (Model #3)")
print("Training accuracy:", train_acc_tree)
print("Test accuracy      :", test_acc_tree)

# Confusion matrix on the test data
print("\nConfusion matrix for Decision Tree (test data):")
print(confusion_matrix(y_test, y_test_pred_tree))

# This plot shows the main splits the tree is using to classify mushrooms.
# max_depth=3 keeps the picture readable, even though the full tree is deeper.
plt.figure(figsize=(20, 10))
plot_tree(
    tree_model,
    feature_names=X_train.columns,

```

```

class_names=["edible", "poisonous"],
filled=True,
rounded=True,
max_depth=3          # only show top levels of the tree
)
plt.title("Decision Tree for Mushroom Classification (truncated at depth 3)")
plt.tight_layout()
plt.show()

```

```

# ----- Naive Bayes (Model #4) -----#

# I am using Naive Bayes as another classification model.
# It predicts poisonous (1) vs edible (0) using a simple "independence"
# assumption between predictors.

nb_model = GaussianNB()

# Fit the model on the training data
nb_model.fit(X_train, y_train)

# Predictions on training and test sets
y_train_pred_nb = nb_model.predict(X_train)
y_test_pred_nb  = nb_model.predict(X_test)

# Classification accuracy on training and test data
train_acc_nb = accuracy_score(y_train, y_train_pred_nb)
test_acc_nb  = accuracy_score(y_test, y_test_pred_nb)

print("\nNaive Bayes (Model #4)")
print("Training accuracy:", train_acc_nb)
print("Test accuracy    :", test_acc_nb)

# Confusion matrix on the test data
print("\nConfusion matrix for Naive Bayes (test data):")
print(confusion_matrix(y_test, y_test_pred_nb))

```

```

# Quadratic Discriminant Analysis (Model #5)

# QDA allows a different covariance matrix for each class.
# This can capture more flexible class boundaries than

```



```

#LDA or logistic regression.

qda_model = QuadraticDiscriminantAnalysis()

# Fit QDA on the training data
qda_model.fit(X_train, y_train)

# Predictions on training and test sets
y_train_pred_qda = qda_model.predict(X_train)
y_test_pred_qda  = qda_model.predict(X_test)

# Classification accuracy on training and test data
train_acc_qda = accuracy_score(y_train, y_train_pred_qda)
test_acc_qda  = accuracy_score(y_test, y_test_pred_qda)

print("\nQuadratic Discriminant Analysis (Model #5)")
print("Training accuracy:", train_acc_qda)
print("Test accuracy    :", test_acc_qda)

# Confusion matrix on the test data
print("\nConfusion matrix for QDA (test data):")
print(confusion_matrix(y_test, y_test_pred_qda))

```

```

# Model #6 -- Bagging (Trees)

# BaggingClassifier in sklearn does bagging of decision trees by default.
# It fits many trees on bootstrap samples and averages their predictions
# to reduce variance.

bag_model = BaggingClassifier(
    n_estimators=200, # number of trees in the ensemble
    random_state=seed, # keep it reproducible (seed = 448)
    n_jobs=-1         # use all cores to speed things up
)

# Fit the bagging model on the training data
bag_model.fit(X_train, y_train)

# Predictions on training and test data
y_train_pred_bag = bag_model.predict(X_train)
y_test_pred_bag  = bag_model.predict(X_test)

```

```
# Accuracy on training and test sets
train_acc_bag = accuracy_score(y_train, y_train_pred_bag)
test_acc_bag  = accuracy_score(y_test, y_test_pred_bag)

print("\nBagging (Model #6)")
print("Training accuracy:", train_acc_bag)
print("Test accuracy    :", test_acc_bag)

# Confusion matrix on the test data
print("\nConfusion matrix for Bagging (test data):")
print(confusion_matrix(y_test, y_test_pred_bag))
```